

Progetto d'esame di
Programmazione e modellazione
ad oggetti

Samuele Caporale - 329096

A.A. 2024/2025

Indice

1	Analisi	3
1.1	Analisi del problema	3
1.2	Analisi dei requisiti	3
1.2.1	Requisiti funzionali	3
1.2.2	Requisiti non funzionali	3
1.3	Analisi del dominio	4
2	Design	5
2.1	Pattern architetturali	5
2.2	Package	5
2.3	Model	6
2.3.1	Diagramma delle interfacce	6
2.4	View e Controller	7
2.4.1	Main	7
2.4.2	LoginController	7
2.4.3	LoginView	8
2.4.4	SessionController	8
2.4.5	SessionView	10
2.4.6	GameController	11
2.4.7	GameView	12
2.5	Model.Cpu	14
3	Sviluppo	15
3.1	Ambiente di Sviluppo	15
3.2	Controllo di Versione e Tracciabilità	15
3.2.1	Repository	15
3.3	Librerie esterne	15
3.4	Workflow	15
3.5	Testing	16
3.5.1	Model	16
3.5.2	View e Controller	16
4	Commenti conclusivi	17

1 Analisi

1.1 Analisi del problema

Lo scopo del progetto consiste nella realizzazione del gioco di decodifica Master Mind, in una sua variante.

L'obiettivo del gioco è indovinare una sequenza segreta di colori, entro un numero finito di tentativi e/o prima dell'avversario.

Ogni tentativo si compone di un numero fisso di colori che dipende dal livello della partita corrente.

Al termine di ogni tentativo viene fornito il relativo numero di colori corretti e di colori corretti in posizione corretta.

Il gioco può essere svolto in due modalità differenti: single player e ai challenge.

Il gioco termina:

- con un fallimento se il giocatore esaurisce i tentativi e/o l'avversario indovina la sequenza segreta.
- con successo se il giocatore indovina la sequenza segreta.

1.2 Analisi dei requisiti

1.2.1 Requisiti funzionali

- Il gioco deve generare una sequenza segreta di colori a seconda del livello.
- Il giocatore deve poter inserire tentativi della lunghezza corretta.
- Il sistema deve fornire un feedback sul numero di colori corretti e posizione corretta.
- Il sistema deve memorizzare le statistiche di gioco in single player.
- Deve essere disponibile la modalità "single player" e la modalità "Ai challenge" contro l'algoritmo.
- L'interfaccia deve consentire il drag-and-drop dei colori sulle celle del tentativo corrente.

1.2.2 Requisiti non funzionali

- L'interfaccia deve essere semplice ed efficace nel mostrare statistiche e modalità di gioco.
- Il costo computazionale per il calcolo algoritmico della cpu dev'essere minore di $O(c^k)$.
- Il sistema deve essere scalabile, per supportare modalità e livelli con sequenze differenti.
- Il codice deve essere manutenibile e documentato per futuri aggiornamenti.

1.3 Analisi del dominio

Il dominio del problema è costituito dall'insieme di tutte le sequenze possibili di colori che possono rappresentare la combinazione segreta.

Sia c il numero di colori disponibili e k la lunghezza della sequenza:

- Per la modalità **SINGLE PLAYER**, nei livelli in cui non sono ammesse ripetizioni, il numero di combinazioni possibili è dato dalle permutazioni:

$$P(c, k) = \frac{c!}{(c - k)!}$$

- Per la modalità **SINGLE PLAYER**, nei livelli in cui sono ammesse ripetizioni, il numero di combinazioni possibili aumenta all'aumentare di c , introducendo così colori errati.
- Per la modalità **AI CHALLENGE** è stata scelta la configurazione classica di $c = 6$ e $k = 6$ senza ripetizioni, per un totale di:

$$6! = 720$$

combinazioni possibili.

Il codominio è l'insieme delle soluzioni corrette, costituito dalla sequenza che corrisponde al codice segreto.

2 Design

2.1 Pattern architetturali

Il pattern di riferimento utilizzato è il Model-View-Controller.

- Il **Model** è stato realizzato in puro Java 21, completamente indipendente dalla GUI.
- La **View** implementata con JavaFX e personalizzata tramite JavaFX CSS.
- Il **Controller** funge da intermediario tra **Model** e **View**, gestendo le azioni dell'utente e aggiornando la **View**. Utilizza sia tipi primitivi e reference di Java 21 sia JavaFX Properties per il binding dei dati.

2.2 Package

L'organizzazione dei package è suddivisa in base al pattern MVC come segue:

```
src/  
├── controller/  
│   ├── GameController.java  
│   ├── LoginController.java  
│   └── SessionController.java  
├── model/  
│   ├── Code.java  
│   ├── CodeImpl.java  
│   ├── Color.java  
│   ├── Cpu.java  
│   ├── CpuImpl.java  
│   ├── Game.java  
│   ├── GameImpl.java  
│   ├── GameState.java  
│   ├── Hints.java  
│   ├── Player.java  
│   ├── Session.java  
│   └── SessionImpl.java  
├── test/  
│   ├── TestCpu.java  
│   ├── TestSession.java  
│   └── TestGame.java  
├── view/  
│   ├── GameView.java  
│   ├── LoginView.java  
│   ├── Main.java  
│   └── SessionView.java  
├── view.assets/  
│   └── style.css  
└── src/resources/
```

2.3 Model

2.3.1 Diagramma delle interfacce

Il Model si compone di 2 entità principali: la **Session** e il **Game**. La **Session** si occupa di gestire il **Player**, lo storico dei punteggi **GameStats** e il **Game** corrente.

Il **Game** rappresenta la singola partita ed utilizza principalmente **Code** per i tentativi e **Hints** per i risultati.

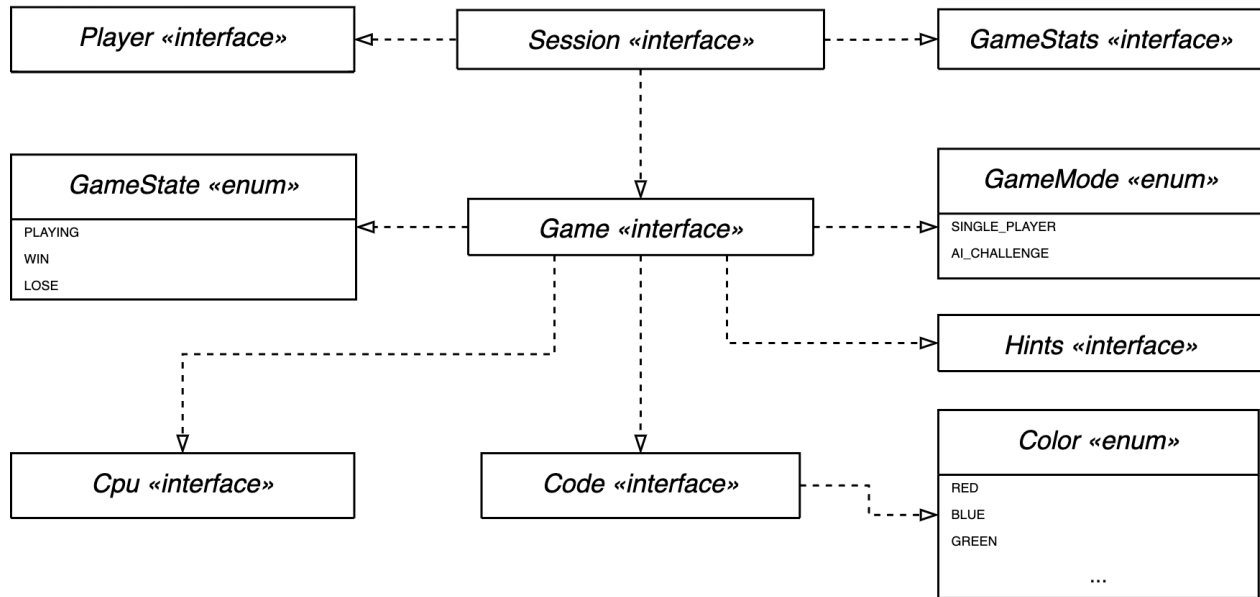


Figura 1: Interface diagram UML (Model)

Vengono utilizzati enumerati per classificare alcuni stati, come **GameMode** per specificare nel costruttore di **Game** quale tipo di partita si intende iniziare, **Color** per la sequenza segreta e per comporre i tentativi in **Code** ed infine **GameState** grazie al quale teniamo traccia dello stato corrente del game nel corso dei tentativi.

Per calcolare i valori delle statistiche in **Session** sono stati utilizzati principalmente **Stream** con **method reference**.

2.4 View e Controller

2.4.1 Main

Il punto da cui si avvia il gioco è Main, classe che estende Application, fa overriding del metodo start occupandosi di creare lo stage principale, la LoginView e LoginController.

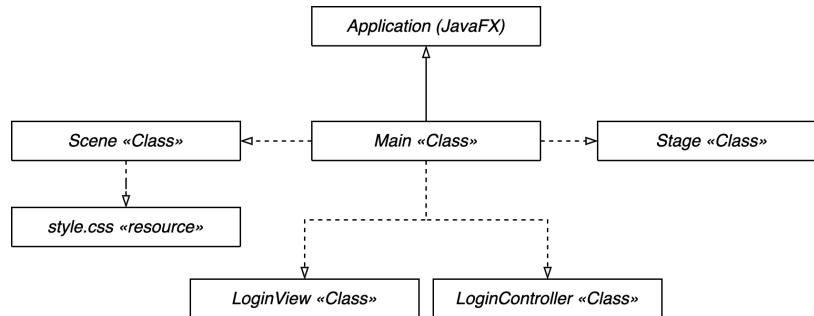


Figura 2: UML Main

Viene inizialmente creata la LoginView, passata successivamente al costruttore del LoginController insieme al nuovo Stage, quest'ultimo servirà per non ricreare una nuova finestra ma sfruttare la stessa modificandone solo dimensioni e contenuto **root**.

2.4.2 LoginController

Il LoginController si occupa, con il suo unico metodo **handleStart** di acquisire il nome del giocatore, creare un nuovo Player, istanziare SessionImpl usando il Singleton e creare il SessionController.

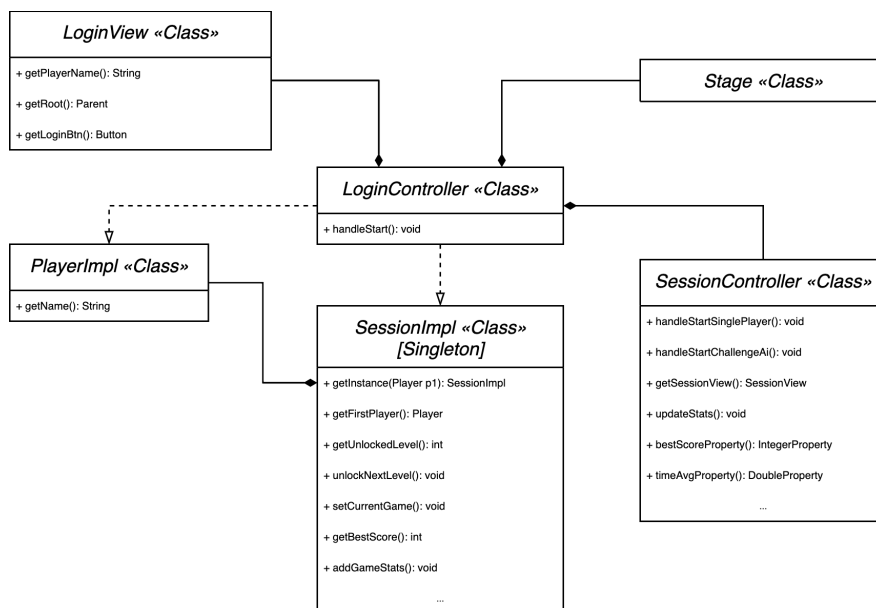


Figura 3: UML LoginController

Come ultimo step, aggiorna la **root** della **Scene** corrente per mostrare la nuova **View**.

2.4.3 LoginView

La Classe **LoginView** si occupa di mostrare il campo di testo per l'acquisizione del nome e il bottone per creare la sessione.



Figura 4: LoginView

2.4.4 SessionController

Creando il **SessionController** al suo interno viene creata la relativa **View**.

Questa classe si occupa di gestire l'intera sessione, mostrare le statistiche e fornire l'accesso ai game. Per non intaccare il model con tipi di JavaFX e quindi renderlo portabile ad un eventuale cambiamento di framework grafico, nel Controller sono stati utilizzati **IntegerProperty** e **DoubleProperty**, con i quali grazie al binding viene mantenuta la view reattiva al cambiamento del model.

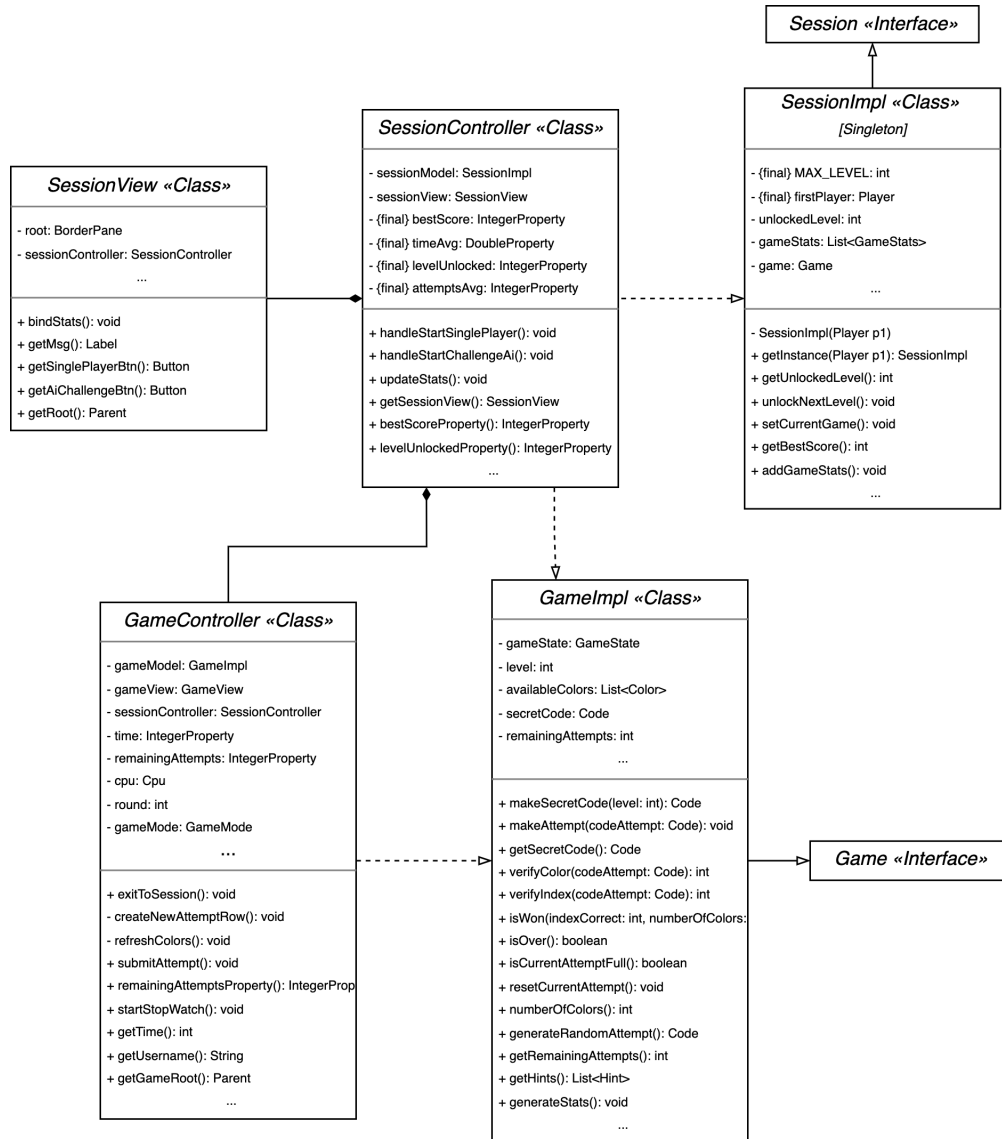


Figura 5: UML SessionController

Questa classe contiene i metodi principali `handleStartSinglePlayer` e `handleStartChallengeAi` per avviare i game in base alla **GameMode** scelta dall'utente, i metodi per fornire i valori delle statistiche ed aggiornarle al termine del game. Inoltre il metodo `getSessionView` che restituisce la **SessionView** da cui si può ricavare la **root** per aggiornare la Scene.

2.4.5 SessionView

La SessionView, oltre a contenere la struttura grafica della sessione fornisce i metodi getter utili al SessionController per accedere ai bottoni e quindi applicare `ActionEvent` o per aggiornare i valori testuali. Fornisce anche un metodo dedicato al binding delle statistiche che collega la view al controller in modo reattivo.

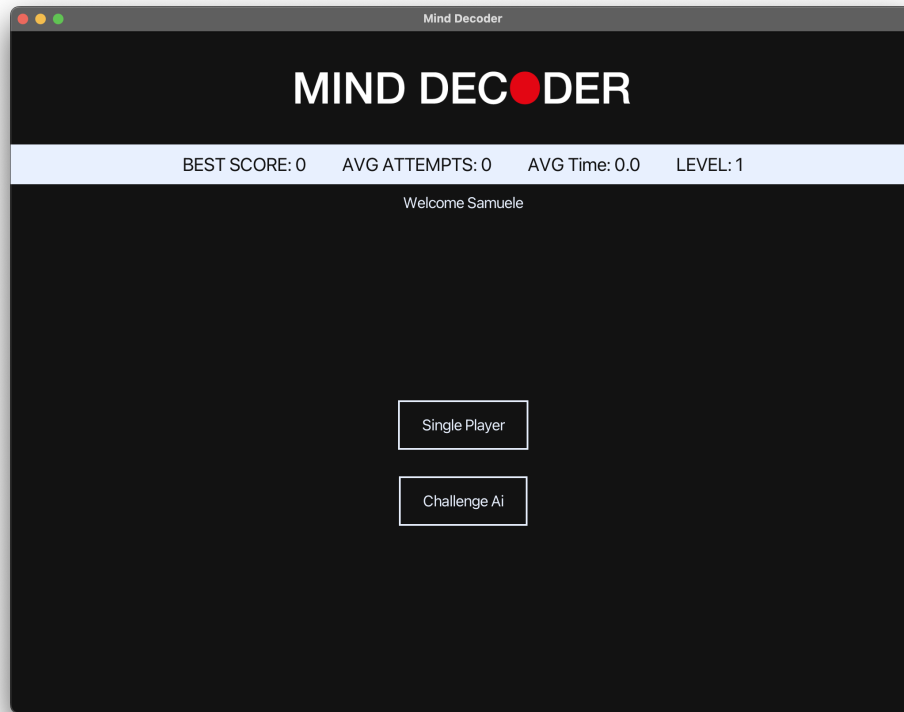


Figura 6: SessionView

2.4.6 GameController

Il GameController coordina il gioco tra modello e interfaccia, gestendo lo stato, i tentativi e tempo. Si occupa di aggiornare la view quando il giocatore o la CPU fanno mosse. Fornisce metodi per inviare tentativi, mostrare suggerimenti e gestire la vittoria o la sconfitta. Agisce come intermediario tra sessione utente e gioco, garantendone il corretto flusso.

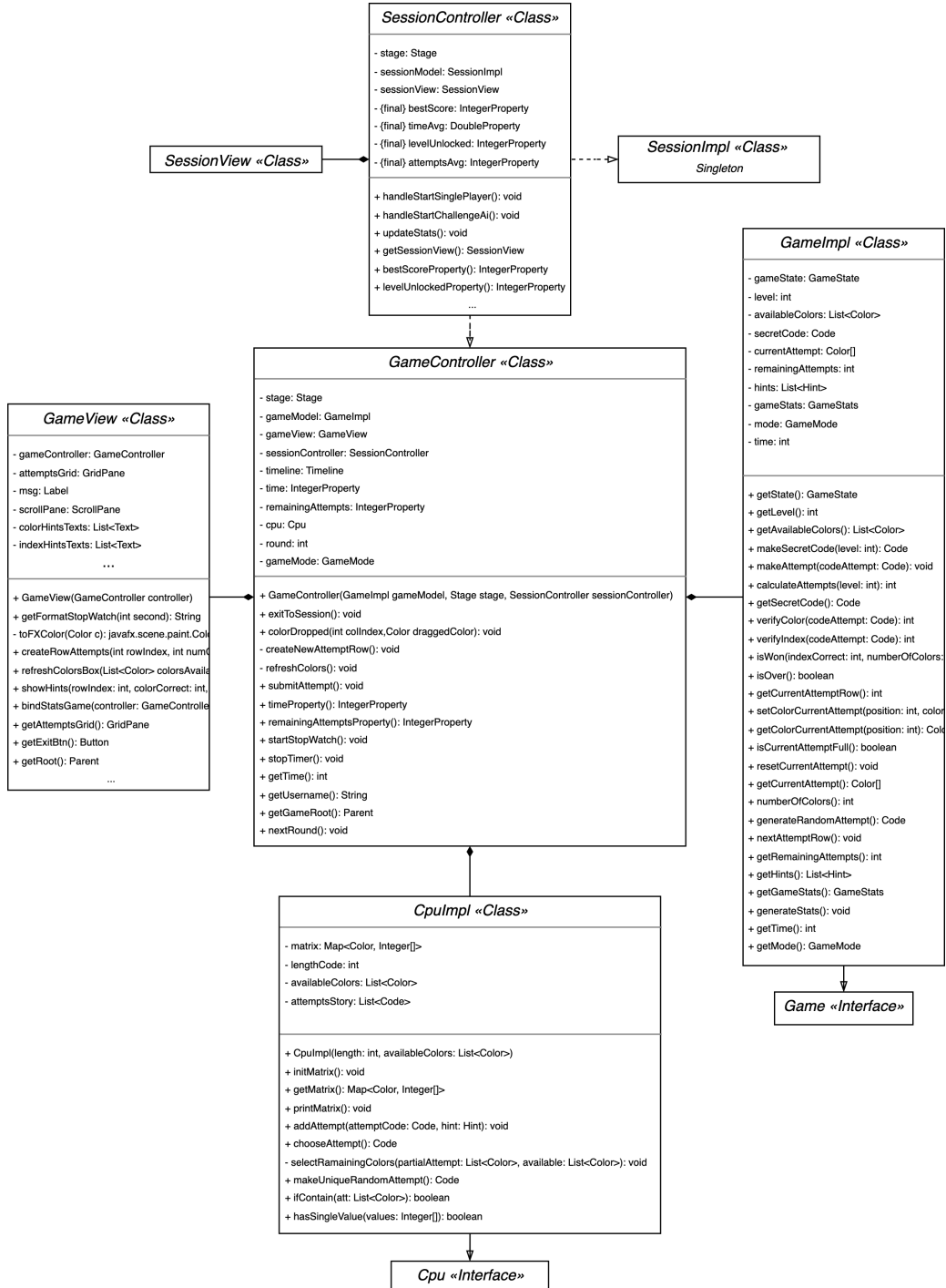


Figura 7: GameController

Il `GameController` utilizza i metodi del `model.Game` e `model.Cpu` per gestire l'intero game.

Per iniziare all'interno del costruttore viene salvato il riferimento al `sessionController` usato dal metodo `exitToSession` per poter tornare a mostrare la `SessionView` all'utente.

Sempre all'interno del costruttore viene creata una variabile intera per gestire i round, e due variabili di JavaFX Properties come precedentemente annunciato con finalità di binding, per `attempts` e `time`.

Nel caso la `GameMode` scelta sia `AI CHALLENGE` verrà anche creata un'istanza di `CpuImpl` ed inizializzata con l'apposito metodo `initMatrix`.

Viene poi fatto partire lo `stopWatch`, si crea la prima riga dei tentativi e si mostrano i colori disponibili all'utente.

Infine si applica il gestore di eventi di tipo Action Event al bottone di uscita della view.

Il core della business logic del game si trova all'interno del metodo `colorDropped`, serve a gestire e coinvolge la creazione del tentativo corrente fino al completamento, e quindi l'invio della sequenza, per entrambi le modalità di gioco; Questo avviene quando il controller intercetta l'evento `setOnDragDropped` dalla `GameView`, ossia quando il giocatore rilascia un colore in una cella valida, fa quindi il set nel model del colore scelto nella posizione scelta sull'array `currentAttempt` popolando il tentativo. In seguito viene poi rimosso il colore dalla lista dei colori disponibili `availableColors`.

La `GameView` si aggiorna con i metodi `updateCell` e `refreshColorsBox`.

Il tentativo è pronto per essere fatto quando `isCurrentAttemptFull` ritorna `true`.

Una volta riempita la sequenza, nel `GameController` tramite il metodo `submitAttempt` viene chiamato il metodo del model `makeAttempt` passandovi un'istanza di `CodeImpl` contenente la sequenza `currentAttempt`.

Si genera così l'`Hint` e si incrementa il contatore di riga dei tentativi nel model.

La condizione per cui si possono continuare ad effettuare tentativi dipende dalla `GameState` fino a quando rimane `PLAYING` si possono effettuare tentativi, altrimenti non vengono più generate righe in `attemptsGrid` ed in caso di `WIN` viene mostrato il messaggio nella zona dei feedback, fermato lo `stopWatch`, incrementato il livello e salvate le statistiche. In caso contrario la medesima cosa senza aumentare livello nè salvare le statistiche.

Infine abbiamo il round della Cpu che si attiva solo se è stata scelta come `GameMode AI CHALLENGE`, giocherà solo nei round pari come verrà spiegato in seguito nella sezione dedicata alla Cpu.

2.4.7 GameView

La `GameView` rappresenta l'interfaccia di gioco.

Contiene il Layout principale `root` ed organizza le varie aree della schermata in 3 sezioni:

1. **Top area** Comprende le statistiche (tentativi rimanenti e tempo) e le impostazioni (username e pulsante di uscita), oltre a un box dedicato ai feedback. Le statistiche sono

collegate tramite binding al GameController, che memorizza i valori come IntegerProperty. Per il timer è stato scelto di mostrare i valori nel formato classico (hh:mm:ss); a tal fine è presente un'apposito metodo che lo formatta come stringa letterale aggiornata ogni secondo.

Il pulsante di uscita sfrutta il riferimento al SessionController passato durante la creazione del GameController, consentendo di terminare il `game` senza salvare i progressi di gioco.

La label dei feedback viene infine aggiornata dal Controller in caso di vittoria o sconfitta.

2. **Center area** Contiene la griglia dei tentativi `attemptsGrid`, inserita all'interno di uno `ScrollPane` per permettere lo scorrimento verticale. Qui il giocatore inserisce i colori tramite drag-and-drop. Ad ogni colore rilasciato, la View aggiorna la cella `Pane` con il colore scelto, Effettuato il tentativo vengono mostrati gli indizi relativi ai colori e alle posizioni corrette, grazie al metodo `showHints`.
3. **Bottom area** Mostra i colori disponibili per la giocata, rappresentati da cerchi colorati. L'utente può trascinare un colore nella griglia dei tentativi, e una volta usato, il colore viene rimosso dalla visualizzazione. Al termine di ogni giocata, i colori vengono ripristinati nuovamente grazie al metodo `refreshColorsBox`, garantendo che sia sempre aggiornata e coerente con lo stato del Model.

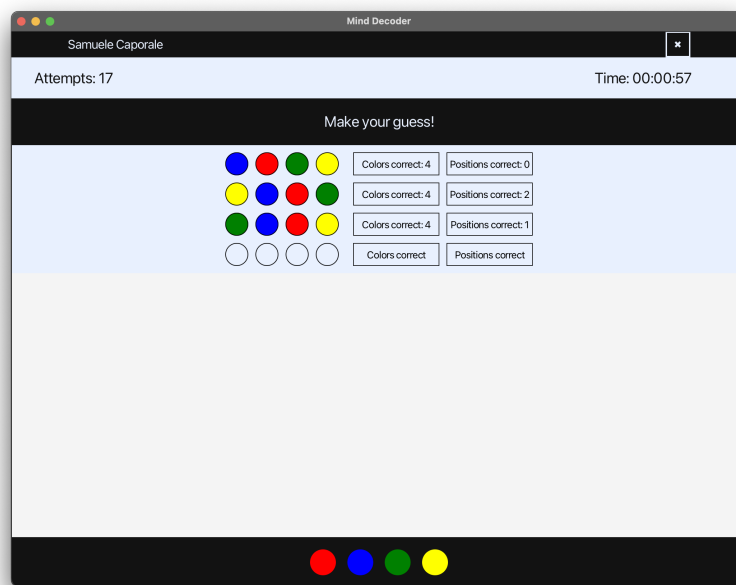


Figura 8: GameView

Fonti:

- Per la griglia è stato preso spunto da: [StackOverflow – GridPane row e column IDs](#)
- Per il drag-and-drop: [Jenkov Tutorials](#) e [Oracle Docs – JavaFX Drag and Drop](#)

2.5 Model.Cpu

La **Cpu** rappresenta l'avversario nella modalità **AI CHALLENGE**. La sua strategia si suddivide in tre fasi distinte.

Nella prima fase viene creata una **HashMap** in cui ogni chiave corrisponde a un **Color**, mentre il valore è un array di dimensione pari alla lunghezza della sequenza. Ogni cella dell'array, inizialmente impostata a zero, rappresenta la probabilità che quel colore si trovi in una determinata posizione.

Nella seconda fase la CPU effettua tentativi pseudo-casuali, evitando duplicati. Ogni nuovo tentativo (sia della CPU che del giocatore) viene memorizzato nella tabella, aggiornando i valori in base al numero di posizioni corrette **indexCorrect** restituito dall'ultimo **Hint**. Quando **indexCorrect** risulta uguale a zero, le posizioni impossibili vengono impostate a **null**, riducendo così progressivamente lo spazio delle possibili soluzioni.

Infine, dopo un certo numero di tentativi, la tabella inizia a mettere in evidenza i colori più probabili per ogni indice dell'array, cioè quelli che hanno contribuito ad aumentare maggiormente il punteggio di **indexCorrect**, ed elimina con certezza le posizioni impossibili. A questo punto la **Cpu** compone la nuova sequenza scegliendo:

- i colori in posizione certa (quando tutti gli altri valori sono **null**);
- i colori con il punteggio più alto per ogni indice della matrice che non genera conflitto o sia già in sequenza come colore certo.

Per evitare duplicati, la **Cpu** gestisce una lista di colori disponibili **availableColors** e ogni colore selezionato viene rimosso man mano che viene inserito nella sequenza.

il tentativo della cpu avviene tramite il metodo **nextRound** del **GameController**, che gestisce la mossa come segue:

- Verifica che la partita sia ancora in **GameState** uguale a **PLAYING**
- Se il **round** è pari esce dal metodo senza fare nulla.
- Se il **round** è dispari ed è ancora nella fase di esplorazione, effettua il tentativo pseudo-casuale unico in modo da distribuire al meglio statisticamente le probabilità sulla matrice.
- Effettua il tentativo tramite il **gameModel**
- Somma nella matrice dei valori il numero di indici corretti ricavato dall'ultimo **Hint** generato tramite i colori nelle posizioni del tentativo appena effettuato.
- Verifica la vittoria ed in caso esce dal metodo aggiornando il feedback.
- Altrimenti incrementa il numero di riga del tentativo e genera una riga vuota pronta per il player.
- Incrementa il round.

3 Sviluppo

3.1 Ambiente di Sviluppo

L'ambiente di sviluppo utilizzato è l'IDE **Eclipse**, versione *2024-09 (4.33.0)*, eseguito su sistema operativo **macOS**. Tale strumento ha fornito un supporto completo per la gestione del progetto, l'organizzazione del codice e il debugging.

3.2 Controllo di Versione e Tracciabilità

L'implementazione è stata gestita tramite il sistema di versionamento distribuito **Git**, che ha permesso di mantenere uno storico dettagliato delle modifiche effettuate. Ogni commit è stato utilizzato per documentare lo stato evolutivo del progetto, favorendo così tracciabilità e reversibilità.

3.2.1 Repository

Il codice sorgente è stato reso disponibile tramite la piattaforma **GitHub**, all'indirizzo: https://github.com/c-samuele/mind_decoder.git

3.3 Librerie esterne

Le librerie esterne utilizzate nel progetto sono le seguenti:

- **JavaFX**
Distribuito sotto licenza **GNU General Public License versione 2 con Classpath Exception**.
Libreria utilizzata per la realizzazione dell'interfaccia grafica (GUI) del progetto, in particolare per la gestione delle scene, dei layout e degli eventi.
- **JUnit 5**
Distribuito sotto licenza **Eclipse Public License 2.0**.
Utilizzato per lo sviluppo e l'esecuzione dei test unitari, al fine di verificare la correttezza delle principali funzionalità implementate.

3.4 Workflow

Il processo di sviluppo ha seguito un approccio incrementale, articolato in fasi: analisi dei requisiti, progettazione delle interfacce principali, implementazione delle funzionalità e successiva integrazione, svolte seguendo pratiche ispirate alla metodologia Agile.

Ogni fase è stata supportata dall'utilizzo di Git per il versionamento, garantendo così una chiara tracciabilità delle attività svolte.

3.5 Testing

3.5.1 Model

Per l'attività di testing è stato utilizzato il framework **JUnit 5**, creando un package dedicato ai test unitari, finalizzati a verificare il corretto funzionamento delle entità più rilevanti del Model e di validare i requisiti funzionali dell'analisi dei requisiti.

La tecnica maggiormente utilizzata è quella delle **classi di partizione**, quindi sono stati testati i limiti superiori, inferiori e i valori centrali.

La Cpu, componente non deterministica, è stata testata ed i suoi test risultano essere Flaky a causa del non determinismo dei tentativi generati.

3.5.2 View e Controller

Per le restanti componenti, è stata adottata una strategia di **End-to-End Testing**, applicata sia durante lo sviluppo che nella fase conclusiva, al fine di validare il comportamento complessivo del sistema.

4 Commenti conclusivi

Nel Model ho utilizzato un'interfaccia per ogni classe in modo da chiarire al meglio i contratti tra le diverse entità.

In diverse situazioni è tornato molto utile l'utilizzo di Stream, method reference, lambda expression per ridurre la complessità ed aumentare la leggibilità del codice.

Lo stopWatch è situato all'interno del Controller, questo perché utilizza tipi di dato dipendenti dal framework grafico, volendo rispettare al meglio il pattern MVC ho deciso di gestirlo lì e aggiornare secondo per secondo il time del Model, così da mantenerlo aggiornato ed a fine game salvare il valore nei GameStats.

Durante le fasi di sviluppo è stato difficile riuscire a dare il giusto peso alle fasi di testing come prevede il modello incrementale adottato, sono stati svolti test mirati temporanei con JUnit o tramite semplici log man mano, ed infinite riprodotti i test di validazione principali per l'accettazione dei requisiti.

Era stata considerata l'implementazione dell'algoritmo di Knuth (five-guess) per la cpu, ma è stato deciso di creare una soluzione algoritmica da zero sullo storico dei tentativi in modo da rendere possibile la vittoria anche al player se impiega meno mosse della cpu, cosa che rimaneva impossibile contro le 5 mosse dell'algoritmo di Knuth, inoltre il limite asintotico computazionale dell'algoritmo five-guess risultava essere $O(c^k)$ contro quello da me implementato circa $O(c^2)$ considerando $c=k$.

Ho trovato interessante quanto complessa la soluzione algoritmica della Cpu per indovinare la sequenza segreta senza rimanere bloccato in massimi locali, La scelta di non utilizzare un'algoritmo già esistente, mi ha portato a riflettere su come ricavare informazione, e ho appreso che una condizione impossibile in alcuni casi da molta più informazione di una condizione parzialmente corretta, cosa che ho utilizzato ampiamente per escludere direttamente circa 6 indici alla volta dalla matrice delle probabilità.